JAVA
PROGRAMMATION OBJET

Centre BTS Al KENDI

2DSI

10. Tableaux et Chaînes de caractères

LES TABLEAUX SONT DES OBJETS

- En Java les tableaux sont considérés comme des objets (dont la classe hérite de Object) :
  - les variables de type tableau contiennent des références aux tableaux
  - les tableaux sont créés par l'opérateur new
  - ils ont une variable d'instance (final) : final int length
  - Java a une syntaxe particulière pour
    - la déclaration des tableaux
    - leur initialisation

DÉCLARATION, CRÉATION ET INITIALISATION DES TABLEAUX

Déclaration :

- la taille n'est pas fixée : int[] tabEntiers ;

Création :

- on doit donner la taille : tabEntiers = new int[5];
- Chaque élément du tableau reçoit la valeur par défaut du type de base du tableau
- La taille ne pourra plus être modifiée par la suite

Initialisation :

- On peut lier la déclaration, la création et l'initialisation d'un tableau
- la longueur est alors calculée automatiquement d'après le nombre de valeurs données (attention, cette syntaxe n'est autorisée que dans la déclaration) :

```
int[] tabEntiers = {8, 2*8, 3, 5, 9};
```

UTILISATION DES TABLEAUX

Affectation en bloc

- On peut affecter "en bloc" tous les éléments d'un tableau avec un tableau anonyme :

```
int[] t;
….
t=new int[] {1, 2, 3};
```

Affectation des éléments

- l'indice commence à 0 et se termine à tabEntiers.length-1

```
tabEntiers[0] =12 ;
```

Taille du tableau

- `int l = tabEntiers.length;`
- `int e = tabEntiers[l] ;`

/*Lève une ArrayIndexOutOfBoundsException */

Déclaration dans une méthode

```
int[] m(String[] t)
```

AFFICHER LES ÉLÉMENTS D'UN TABLEAU

- La méthode toString() héritée de Object du tableau sans modification, n'affiche pas les éléments du tableau

- Une simple boucle, suffit de les afficher

```
class Arguments {
   public static void main(String[] args) {
            for (int i=0; i < args.length; i++)
          System.out.println(args[i].toString());
   }
}
```

```
java Arguments amoussa aissa taha
affichera
Amoussa
aissa
taha
```

COPIER UNE PARTIE D'UN TABLEAU DANS UN AUTRE

- On peut copier les éléments un à un mais la classe System fournit une méthode static plus performante :

tableau source

indice du 1er élément copié

tableau destination

```
public static void
        arraycopy(Object src, int src_position, Object dst,
                                    int dst_position, int length)
```

indice de dst où sera copié le 1er élément

Nombre d'éléments copiés

COMPARAISON DES TABLEAUX

- On peut comparer l'égalité de 2 tableaux (au sens où ils contiennent les mêmes valeurs) en comparant les éléments un à un

```
public class tableaux {

  public static void main(String[] args) {
    int [] tab1; tab1= new int[]{1,2,3};
    int [] tab2; tab2= new int[]{1,4,3};
    int i=0;
    while (i<tab1.length) {
            if (tab1[i] != tab2[i]) break;
            i++;
    }
    if (i==tab1.length) System.out.println("les tableaux sont égaux ");
    else System.out.println("les tableaux sont différents");
  }
```

COMPARAISON DES TABLEAUX

- On peut aussi utiliser les méthodes à 2 arguments de type tableau de la classe Arrays : java.util.Arrays.equals( )

- par exemple : java.util.Arrays.equals(int[] a1, int[] a2)

```
public class tableaux {

public static void main(String[] args) {
    int [] tab1;
    tab1= new int[]{1,2,3};
    int [] tab2;
    tab2= new int[]{1,2,3};
    int i=0;
    if (java.util.Arrays.equals(tab1,tab2))
                System.out.println("les tableaux sont égaux");
    else System.out.println("les tableaux sont différents");
}
```

- Ne pas utiliser la méthode *equals* héritée de *Object*

TABLEAUX À PLUSIEURS DIMENSSIONS

Déclaration

- int[][] notes;
- Chaque élément du tableau contient une référence vers un tableau

Chacun des 30 étudiants a au plus 3 notes

Création

- notes = new int[30][3];
- notes = new int[30][];

Chacun des 30 étudiants a un nombre de notes variable

Il faut donner au moins les premières dimensions

Déclaration, création et initialisation

```
int[][] notes = {  {10, 11, 9} // 3 notes
                   {15, 8}     // 2 notes
                   ... } ;
```

Affectation

```
notes[10][2] = 12;
```

CHAÎNES DE CARACTÈRES

INTRODUCTION

- 3 classes du paquetage java.lang :
  - String pour les chaînes constante
  - StringBuilder ou StringBuffer pour les chaînes variables
  - On utilise le plus souvent String, sauf si la chaîne doit être fréquemment modifiée

LA CLASSE STRING

- Le type de données String (chaîne de caractères) n'est pas un type élémentaire en Java, c'est une classe

- une chaîne de type String est un objet qui n'est utilisable qu'à travers les méthodes de la classe String

- Pour accéder à la classe String et à toutes ses méthodes, on doit importer le package suivant : import java.lang.String ;

- Un littéral de chaîne est une suite de caractères entre guillemets. " abcdef " est un exemple de littéral de String

- les variables de type String bénéficient d'un statut d'utilisation aussi souple que celui des autres types élémentaires

DÉCLARATION, INITIALISATION ET LONGUEUR

- **Déclaration**

```
String str1 ;
```

- **Déclaration avec initialisation**

```
String str1 = "abcdef";
```

Ou

```
String str1 = new String("abcdef");
```

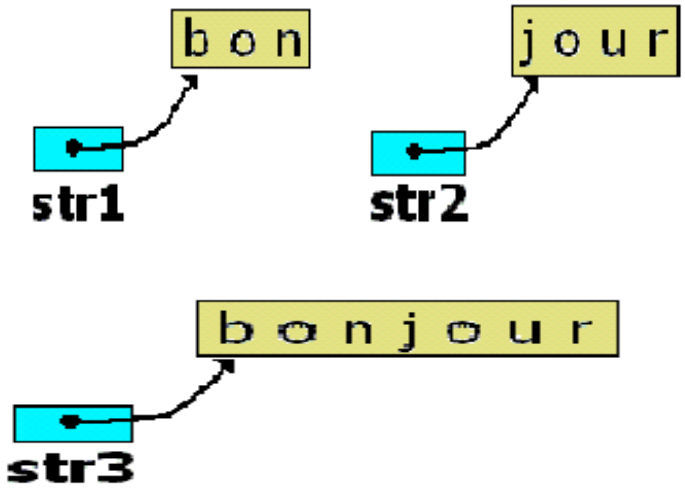
- **Longueur d'une chaîne** : **int** length()

```
String str1 = "abcdef";
int longueur;
longueur = str1.length( ); // ici longueur vaut 6
```

UTILISATION DE MÉTHODE CLASSIQUE SUR LES String

Concaténation de deux chaînes

- Opérateur : + sur les chaînes
- Méthode : String concat(String s)




```
String str1, str2, str3;
str1="bon";
str2="jour";
str3=str1+str2;
```

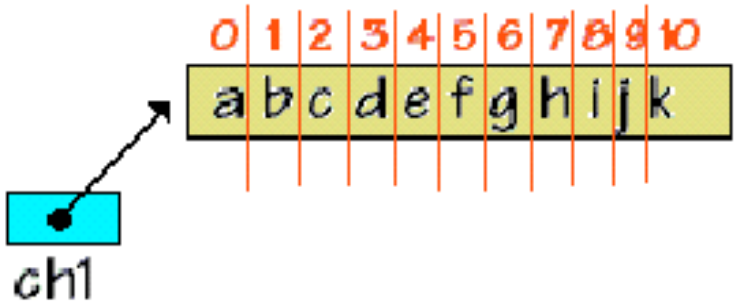
- Ces deux instructions sont équivalentes en Java :

str3 = str1+str2 ⟺ str3 = str1.concat(str2)

UTILISATION DE MÉTHODE CLASSIQUE SUR LES String

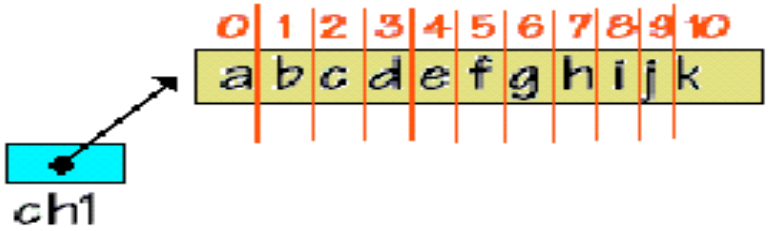
- On accède à un caractère de rang fixé d'une chaîne par la méthode : char charAt(int rang)

```
String ch1 = "abcdefghijk";
char car = ch1.charAt(4);
  // ici car contient la lettre 'e'
```



- Position d'une sous-chaîne à l'intérieur d'une chaîne donnée par la méthode : int indexOf (String ssch)

```
String ch1 = "abcdef" , ssch="cde";
int rang ;
rang = ch1.indexOf(ssch);
  // ici la variable rang vaut 2
```



CONVERSION D'UNE CHAÎNE EN UN TABLEAU

- Les String Java ne peuvent pas être considérées comme des tableaux de caractères

- Pour convertir une chaîne en un tableau de caractères, on doit utiliser la méthode toCharArray

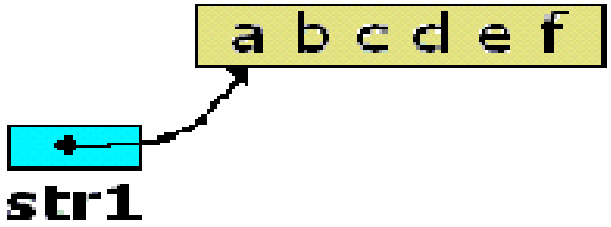
- Attention :
  - Les méthodes de manipulation d'une chaîne ne modifient pas la chaîne objet qui invoque la méthode
  - mais renvoient un autre objet de chaîne différent

```
String str1 = "abcdef" ;
char [ ] tCarac ;
tCarac = str1.toCharArray( ) ;
tCarac = "abcdefghijk".toCharArray( );
```

CONVERSION D'UNE CHAÎNE EN UN TABLEAU

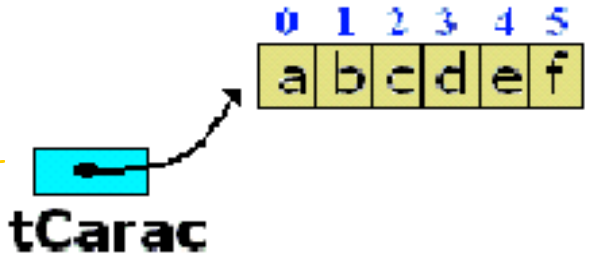
```
String str1 = "abcdef" ;
```

str1 référence un objet de chaîne



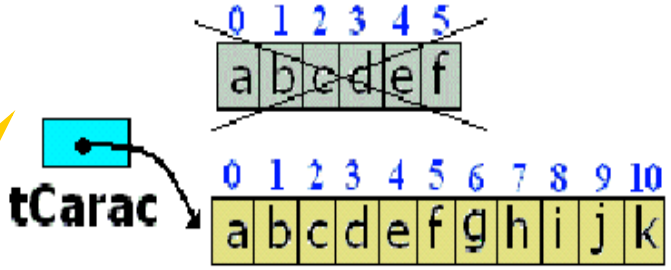

```
char [ ] tCarac ;
tCarac = str1.toCharArray( ) ;
```

tCarac référence un objet de tableau à 6 éléments




```
tCarac = "abcdefghijk".toCharArray( );
```

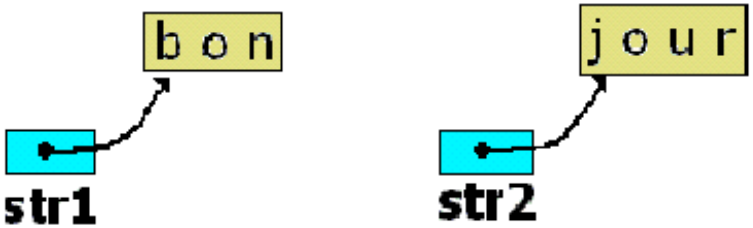
tCarac référence maintenant un nouvel objet de tableau à 11 éléments, l'objet précédent est perdu (éligible au Garbage collector)



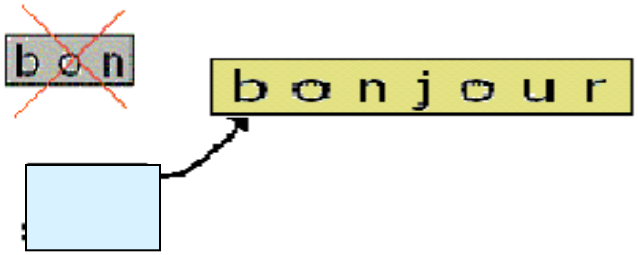
CONVERSION D'UNE CHAÎNE EN UN TABLEAU

- L'exemple précédent sur la concaténation ne permet pas de voir que l'opérateur + ou la méthode concat renvoie réellement un nouvel objet

```
String str1, str2, str3;
str1="bon";
str2="jour";
```




```
str1=str1+str2;
```



un nouvel objet de chaîne a été créé et str1 "pointe" maintenant vers lui.

OPÉRATEURS D'ÉGALITÉ DE String

L'opérateur d'égalité = =

- **détermine si deux objets String spécifiés ont la même référence et non la même valeur**

```
String a , b ;
(a == b )
```

renvoie true si les variables a et b référencent chacune le même objet de chaîne sinon il renvoie false.

La méthode boolean equals(Object s)

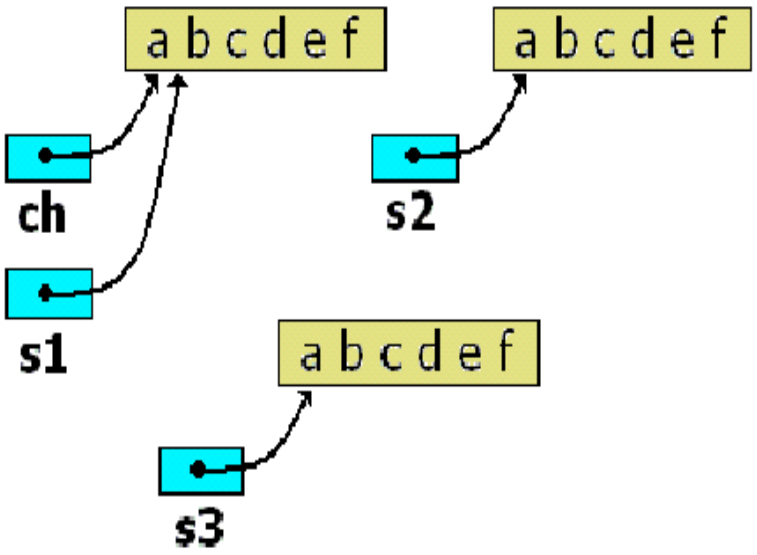
- **teste si deux chaînes n'ayant pas la même référence ont la même valeur.**

```
String a , b ;
a.equals (b)
```

renvoie true si les variables a et b ont la même valeur sinon il renvoie false.

OPÉRATEURS D'ÉGALITÉ DE String

➢ **Exemple :**




```
String s1,s2,s3,ch;
ch = "abcdef";
s1 = ch;
s2 = "abcdef";
s3 = new String("abcdef".toCharArray());
System.out.println("s1="+s1);
System.out.println ("s2="+s2);
System.out.println ("s3="+s3);
System.out.println ("ch="+ch);
if( s1==ch ) System.out.println ("s1=ch"); else System.out.println ("s1<>ch");
if( s1==s3 ) System.out.println ("s1=s3"); else System.out.println ("s1<>s3");
if( s1.equals(s2) ) System.out.println ("s1 même val. que s2");
                                else System.out.println ("s1 différent de s2");
if( s1.equals(s3) ) System.out.println ("s1 même val. que s3");
                                else System.out.println ("s1 différent de s3");
if( s1.equals(ch) ) System.out.println ("s1 même val. que ch");
                                else System.out.println ("s1 différent de ch");
```

OPÉRATEURS D'ÉGALITÉ DE String

- Après exécution on obtient :

```
s1=abcdef
s2=abcdef
s3=abcdef
ch=abcdef
s1=ch
s1<>s3
s1 même val. que s2
s1 même val. que s3
s1 même val. que ch
```

RAPPORT ENTRE String ET char

- Une chaîne String contient des éléments de base de type char comment passe-t-on de l'un à l'autre type?

- On ne peut pas considérer un char comme un cas particulier de String, le transtypage suivant est refusé :

```
char car = 'r';
String s;
s = (String)car;
```

- Il faut utiliser la méthode de conversion valueOf des String

```
s = String.valueOf(car);
```

RAPPORT ENTRE String ET char

- On peut concaténer avec l'opérateur +, des char à une chaîne String déjà existante et affecter le résultat à une String

```
String s1 , s2 ="abc" ;
char c = 'e' ;
s1 = s2 + 'd' ;
s1 = s2 + c ;
```

-

L'écriture suivante sera refusée :	Ecriture correcte associée :
String s1 , s2 ="abc" ; **char** c = 'e' ; s1 = 'd' + c ; *// types incompatibles* s1 = 'd' + 'e'; *// types incompatibles*	**String** s1 , s2 ="abc" ; **char** c = 'e' ; s1 = "d" + **String**.valueOf (c) ; s1 = "d" + "e";

- Le caractère 'e' est de type char
- La chaîne "e" est de type String (elle ne contient qu'un seul caractère)

CLASSE StringBuffer

- Il s'agit de créer des objets buffer ou zone de travail qui contiennent des caractères

- On réserve un buffer simplement en indiquant la taille ou la valeur initiale, comme :

```
str=StringBuffer (128) ;
str=StringBuffer ("Bonjour") ;
```

- La différence avec un objet String est que la taille d'un StringBuffer est variable

- Une telle chaîne peut être modifiée et ainsi être utilisée lors d'une construction dynamique

CLASSE StringBuffer

Méthodes spécifiques

- append(ch) ajoute la chaîne ch à la fin de l'actuelle chaîne
- setLength(n) fixe la nouvelle longueur à n, ce qui peut tronquer la chaîne actuelle

Remarque :

- La concaténation + entre String est en fait implémentée en utilisant StringBuffer et la méthode toString()
- Par exemple, ch étant un objet String : ch = ch + "abc" signifie

```
StringBuffer str = new StringBuffer(ch);
str=str.append("abc"); // la chaine "abc" est ajoutée à la fin de str
ch=str.toString();  // str est transformée en chaîne de type String
```

CLASSE StringBuffer

Exemple

```
class InverseChaine {
   public static String inverser(String chaine) {
   int len = chaine.length();

          /** l'objet dest est déclaré instance de la classe
          StringBuffer. L'opérateur new lui alloue de la mémoire et
        StringBuffer(len) l'initialise.*/

  StringBuffer dest = new StringBuffer(len);
  for (int i = (len - 1); i >= 0; i--) {
    dest.append(chaine.charAt(i));
  }
  return dest.toString();
 }
}
```

CLASSE StringBuffer

Suite de l'exemple

```
public class teste_inverse {
      public static void main(String[] args) {
      String ch, ch1;
      ch=new String(" ** Aziz ** ");
      ch1= InverseChaine.inverser(ch);
      System.out.println(" chaîne= " + ch + " chaîne inverse=" + ch1 );
    }
}
```

EXERCICE TABLEAU

L'objectif est de définir une classe capable de gérer un tableau trié d'objets, dès que ces objets appartiennent à une classe implémentant l'interface Ordonnable.

On construira :

Une interface de nom Ordonnable contenant uniquement les deux prototypes :

boolean plusGrand(Object O); Lorsque cette méthode sera implémentée, elle rendra true si l'instance courante doit être considérée comme plus grande que l'instance indiquée dans la parenthèse, et false sinon (peu importe pour le cas de l'égalité)

boolean equals(Object O);
Lorsque cette méthode de la classe Object sera implémentée (redéfinie), elle rendra true si l'instance courante est égale à l'instance indiquée dans la parenthèse, et false sinon.

EXERCICE TABLEAU

Une classe TableauTrie gérant un tableau d'objets Ordonnable(s) qui reste toujours trié par rapport à la relation définie par la méthode plusGrand, par ordre croissant. Cette classe devra contenir au moins :

- **une méthode inserer pour insérer une instance de Ordonnable dans le tableau (on peut augmenter la capacité du tableau par la valeur increment lorsque le tableau est plein *taille ==capacite*)**
- **une méthode supprimer pour supprimer une instance de Ordonnable du tableau. (on peut diminuer la capacité du tableau après suppression par la valeur increment lorsque *taille = =capacite-increment*)**
- **une méthode toString qui redéfinie la méthode toString de Object.**

La classe TableauTrie devra encore contenir les quatre attributs suivants :

- **capacite de type entier représentant le nombre maximum d'objets du tableau**
- **taille de type entier représentant le nombre d'objets courant du tableau**
- **increment de type entier représentant la valeur entière pour augmenter la capacité du tableau**
- **table représente un tableau d'objets Ordonnable(s)**

EXERCICE TABLEAU

Une classe EntiersOrdonnables qui implémente la classe Ordonnable :

- cette classe représente essentiellement un int pour lequel les deux méthodes plusGrand et equals devront être implémenté ;
- elle doit donc contenir en particulier un attribut privé de type int.

Une classe EssaiTableauTrie, qui contienne la méthode main qui prend en arguments une liste d'entiers et qui :

- met chaque entier dans une instance d'EntiersOrdonnables
- insère les instances d'EntiersOrdonnables dans une instance de TableauTrie
- affiche les entiers par ordre croissant.
- demande un entier à supprimer, et le supprime s'il existe (utiliser la classe Clavier.java)
- affiche la liste triée des entiers restants.

EXERCICE TABLEAU

```
interface Ordonnable {
  boolean plusGrand(Object O);
  boolean equals(Object O);
}

  class TableauTrie {
    private int capacite;
    private int increment;
    private Ordonnable[] table;
    private int taille=0; // pour le nombre de Ordonnable  contenus dans table

    public TableauTrie() {
                int capacite=10;
                int increment=3;
                table=new Ordonnable[capacite];
   }

    public TableauTrie(int capacite, int increment) {
      this.capacite=capacite;
      this.increment=increment;
      table=new Ordonnable[capacite];
   }
```

EXERCICE TABLEAU

```
void inserer(Ordonnable element) {
    int i;
    if (taille==capacite) { // on agrandit le tableau
        capacite+=increment;
        Ordonnable[] grandTableau=new Ordonnable[capacite];
        for (i=0;i<taille;i++)grandTableau[i]=table[i];
        table=grandTableau;
    }
    i=taille-1;
    while ((i>=0) && (table[i].plusGrand(element))) {
        table[i+1]= table[i];
        i--;
    }
    table[i+1]=element;
    taille++;
}
```

EXERCICE TABLEAU

```
void supprimer(Ordonnable element) {
   int i=0;
   while((i<taille)&&(!(element.equals(table[i])))) i++;
   if (i<taille) {
      while (i<taille-1) {
                              table[i]=table[i+1];
                              i++;  }
      taille--;
      if (taille==capacite-increment) { //on diminue le tableau
          capacite-=increment;
          Ordonnable[] petitTableau=new Ordonnable[capacite];
          for (i=0;i<taille;i++) petitTableau[i]=table[i];
          table=petitTableau; }
    }
 }

public String toString() {
   String chaine="";
   for (int i=0;i<taille;i++) chaine+=table[i]+" ";
   chaine+="\n";
   return chaine;  }
}
```

EXERCICE TABLEAU

```
class EntiersOrdonnables implements Ordonnable {
  private int entier;

  EntiersOrdonnables(int nombre) {
    entier=nombre;
  }

   public boolean equals(Object O) {
    // if (!(O instanceof EntiersOrdonnables)) return false;
    return ((EntiersOrdonnables)O).entier==entier;
  }

   public boolean plusGrand(Object O) {
    return entier>((EntiersOrdonnables)O).entier;
  }

  public String toString() {
    return Integer.toString(entier);
  }
}
```

EXERCICE TABLEAU

```
import java.io.*;

public class EssaiTableauTrie  {
  public static void main (String[] argv) throws IOException {
  int s=0;
  TableauTrie tableau =  new TableauTrie(argv.length,1);

  for(int i=0;i<argv.length;i++) {
    try  { s=Integer.parseInt(argv[i]);
            tableau.inserer(new EntiersOrdonnables(s));}
    catch (NumberFormatException e) { System.out.println(argv[i]+" n'est pas entier"); }
  }

 System.out.println(tableau);

 System.out.print("donnez un entier à supprimer : ");

 try  {   s=Clavier.saisie_int();
          tableau.supprimer(new EntiersOrdonnables(s));}
 catch (NumberFormatException e) {System.out.println("Vous n'avez pas tapé entier“ ); }

 System.out.println(tableau);
  }
}
```